



Reinforcement Learning II

Course AIAA 4220

Week 9- Lecture 18

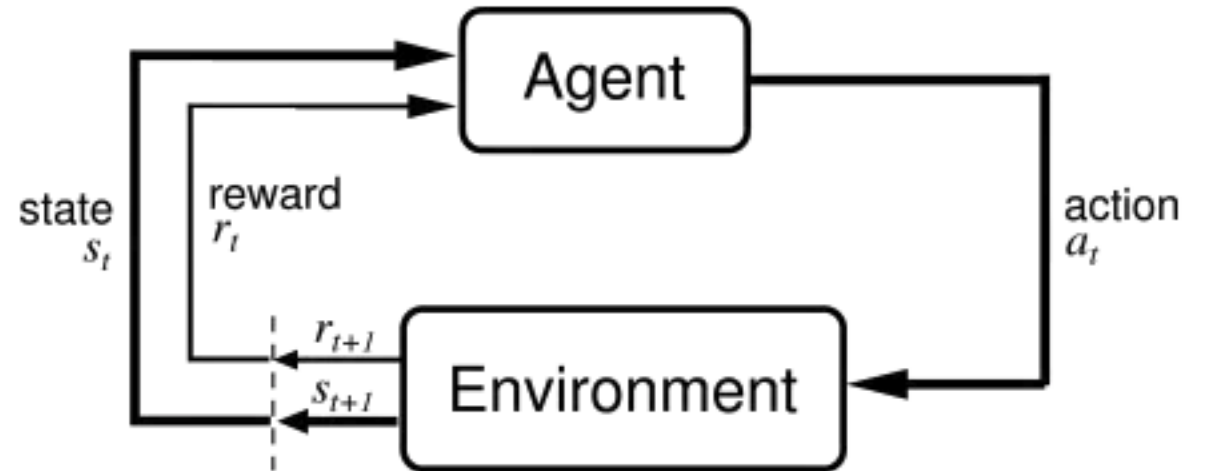
Changhao Chen

Assistant Professor

HKUST (GZ)

RL vs. Supervised Learning

- Reinforcement Learning
 - Sequential decision making
 - Maximize cumulative reward
 - **Sparse supervision**
 - Environment maybe unknown
- Supervised Learning
 - One-step decision making
 - Maximize immediate reward
 - Dense supervision
 - Environment always known



Intersection between RL and supervised learning

- **Imitation learning**

- Obtain expert trajectories (e.g. human driver/video demonstrations):

$$s_0, a_0, r_0, s_1, a_1, r_1, s_2, a_2, r_2, \dots$$

- Perform supervised learning by predicting expert action
- Example - motion planning with expert demonstrations:
 - Lift, Splat, Shoot: Encoding Images from Arbitrary Camera Rigs by Implicitly Unprojecting to 3D (LSS), ECCV'20
 - NEAT: Neural Attention Fields for End-to-End Autonomous Driving, ICCV'21



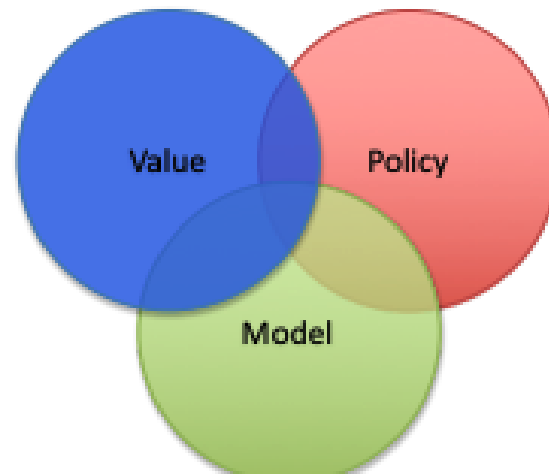
End-to-end autonomous driving
with imitation learning

- **Problem**

- Distribution mismatch between training and testing
- Hard to recover from sub-optimal states
- Sometimes not safe/possible to collect expert trajectories

Major Components of an RL Agent

- An RL agent may include one or more of these components:
 - Policy: agent's behaviour function
 - Decision-making function (control strategy) of the agent, which represents **a mapping from states to actions**
 - Value function: how good is each state and/or action
 - Mapping from states to real numbers, where the value of a state represents the long-term reward achieved starting from that state, and executing a particular policy
 - Model: agent's representation of the environment
 - The agent's view of the environment, which maps state-action pairs to probability distributions over states.
 - **Not every reinforcement learning agent uses a model of its environment**



Policy

- A policy is the agent's behaviour
- It is a mapping from state to action, e.g.
 - Deterministic policy: $a = \pi(s)$
 - Stochastic policy: $\pi(a|s) = \mathbb{P}[A_t = a|S_t = s]$

Value Function

- Value function is a prediction of future reward
- Used to evaluate the goodness/badness of states
- And therefore to select between actions, e.g.

$$v_{\pi}(s) = \mathbb{E}_{\pi} [R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots \mid S_t = s]$$

Expectation of the future cumulative rewards

Model

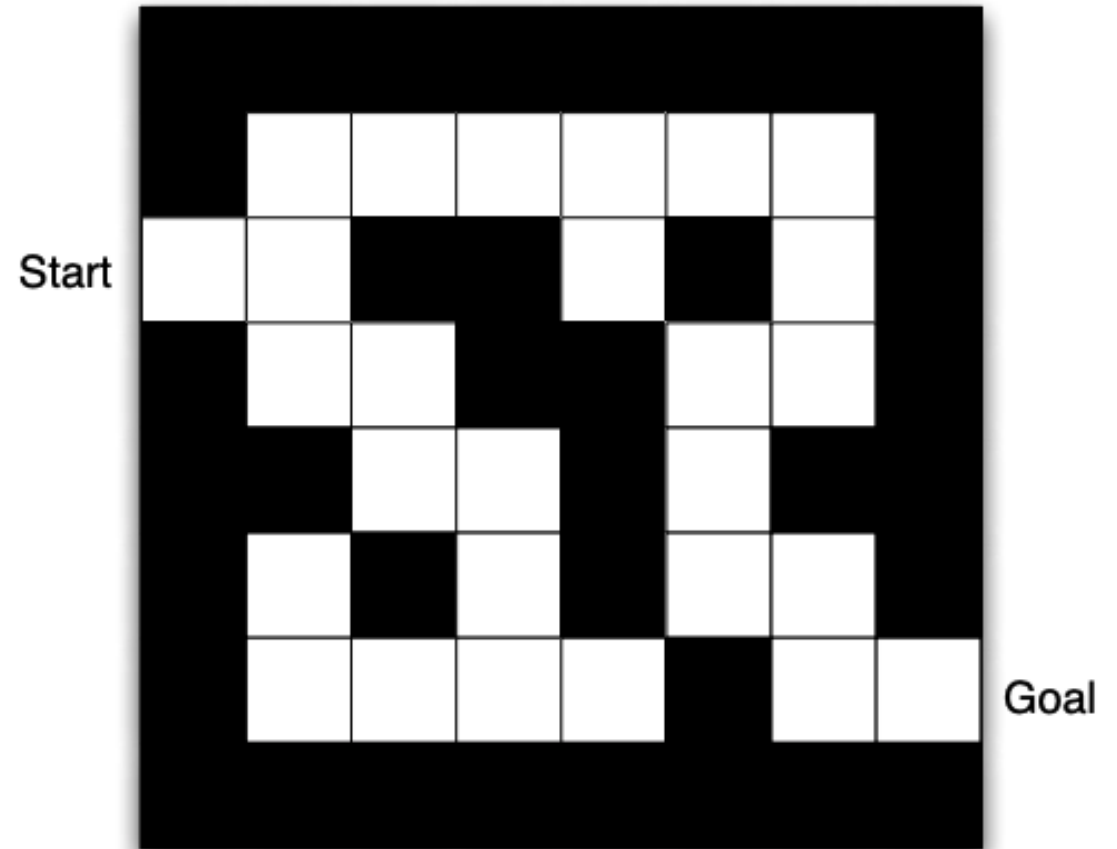
- A model predicts what the environment will do next
 - Namely, the **state transition model** and **reward function**
- $\mathcal{P}$ predicts the next state
 - Previously in MDP it is represented as: $T(s, a, s')$
- $\mathcal{R}$ predicts the next (immediate) reward

$$\mathcal{P}_{ss'}^a = \mathbb{P}[S_{t+1} = s' \mid S_t = s, A_t = a]$$

$$\mathcal{R}_s^a = \mathbb{E}[R_{t+1} \mid S_t = s, A_t = a]$$

Maze Example

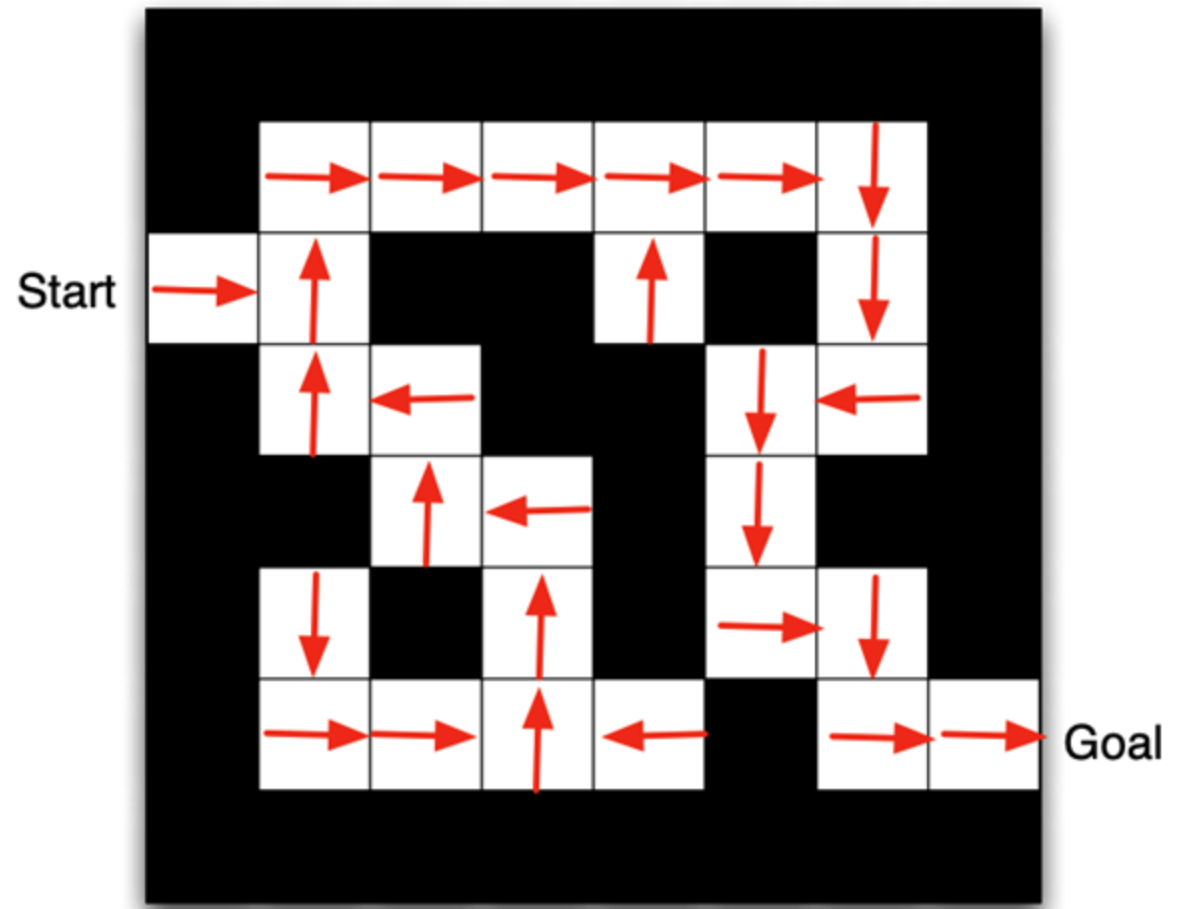
- Rewards: -1 per time-step
- Actions: N, E, S, W
- States: Agent's location



Maze Example

Policy

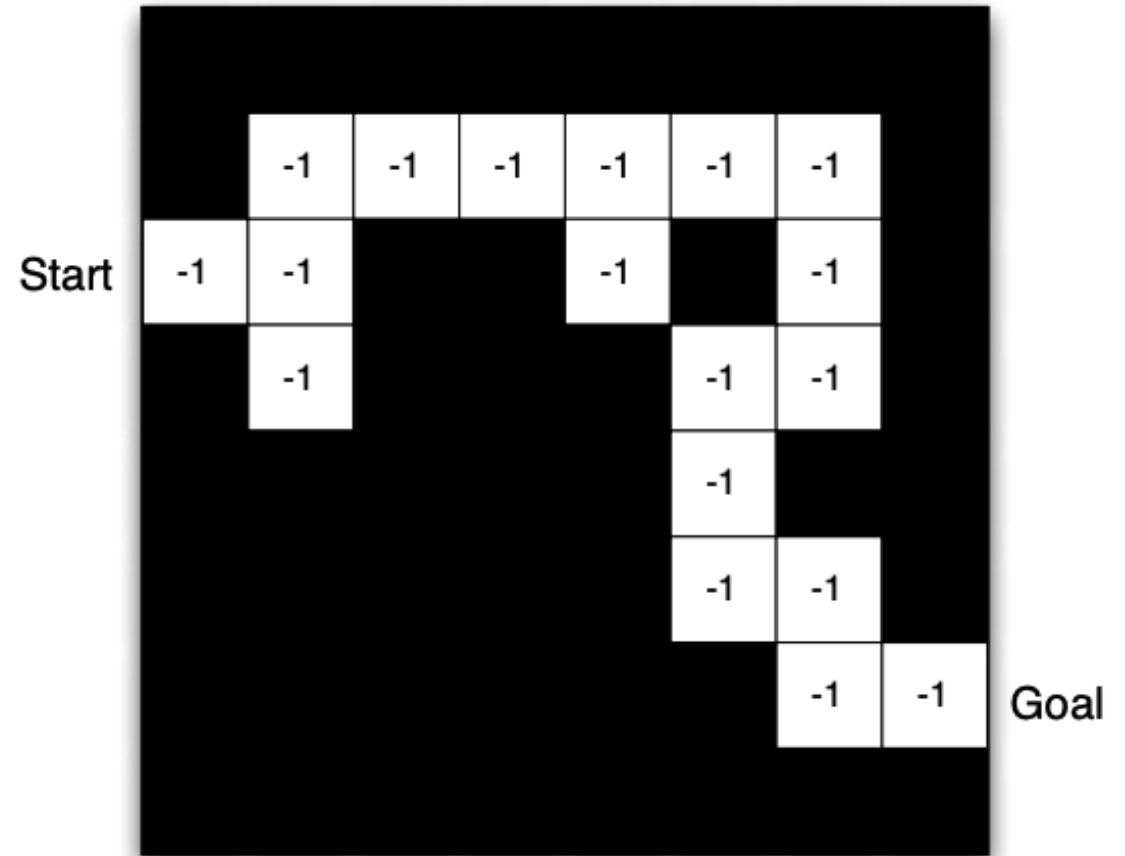
- Arrows represent policy $\pi(s)$ for each state s



Maze Example

Model

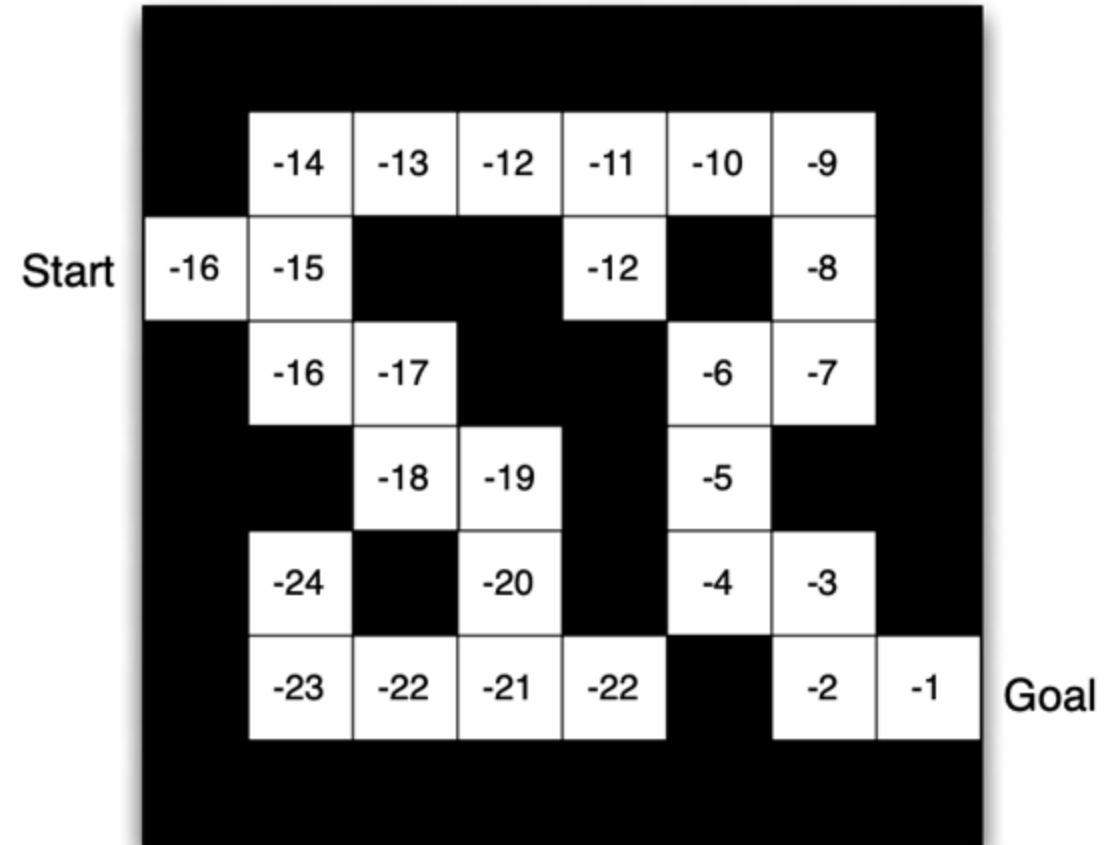
- Agent may have an internal model of the environment
- Dynamics: how actions change the state
- Rewards: how much reward from each state
- The model may be imperfect
- Grid layout represents transition model
- Numbers represent $\mathcal{P}_{ss'}^a$ **immediate reward** from each state s ($\mathcal{R}_s^a$ for all a)



Maze Example

Value Function:

- Numbers represent value $v_{\pi}(s)$ of each state s



Relationships between state and action values

State value functions

Summing over all actions with Q value function gives you the state value

$$V^{\pi}(s) = \sum_a \pi(a|s) Q^{\pi}(s, a)$$

$$V^*(s) = \max_{\pi} V^{\pi}(s)$$

Maxing over all value gives you the optimal value

$$V^{\pi}(s)$$

$$V^*(s)$$

Action value functions

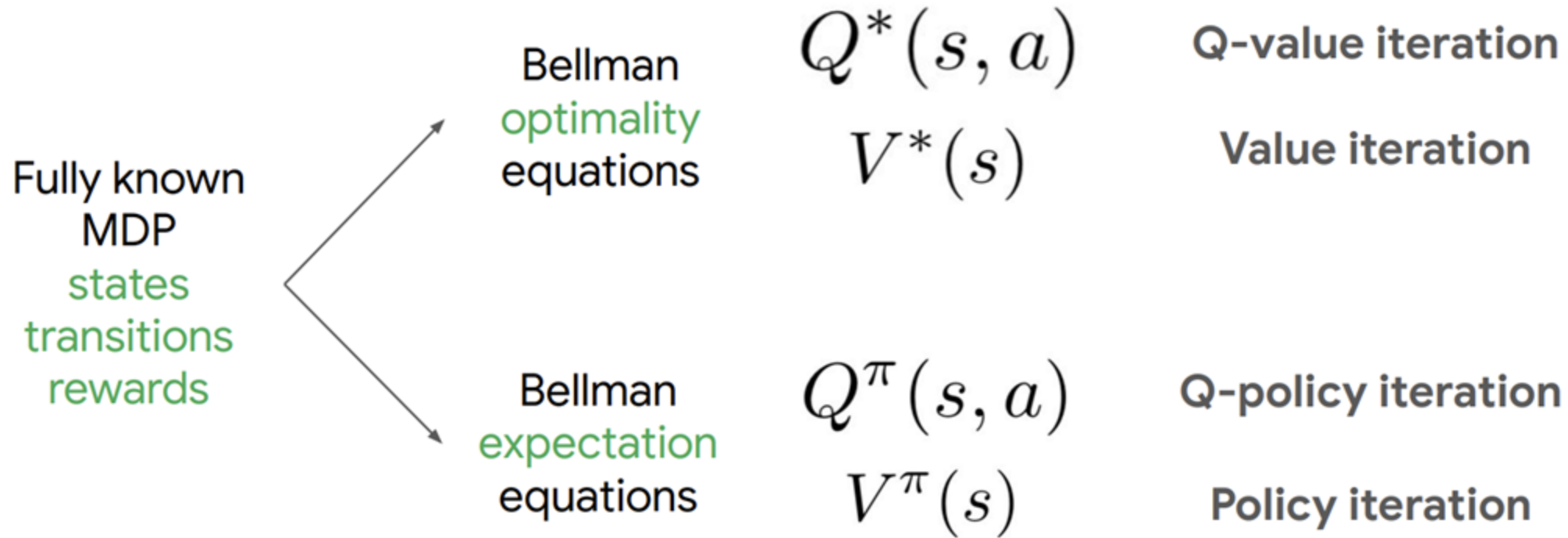
$$Q^{\pi}(s, a)$$

$$Q^*(s, a) = \max_{\pi} Q^{\pi}(s, a)$$

$$Q^*(s, a)$$

$$V^*(s) = \max_a Q^*(s, a)$$

Summary - Solving MDPs using Exact Methods



Repeat until convergence. Guaranteed to converge to optimal policy

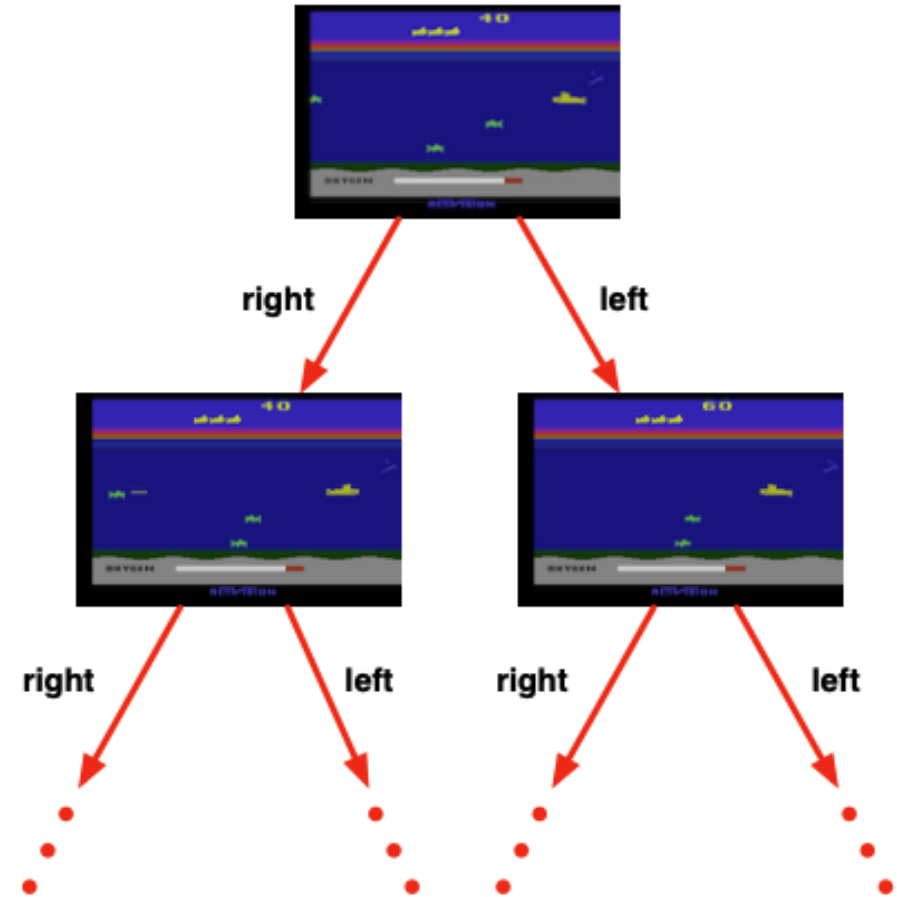
Limitations: Iterate over and storage for all states and actions: requires small, discrete state and action space. Update equations require fully observable MDP and known transitions

From Planning to Learning

- In previous lecture on MDP, we've focused on planning
 - Where the environment and the dynamics are known
 - We can then solve MDP just by computation (without any external interaction)
- For reinforcement learning
 - Environment is initially unknown
 - We need to act to figure it out
- Important ideas in reinforcement learning that came up
 - Exploration: you have to try unknown actions to get information
 - Exploitation: eventually, you have to use what you know
 - Sampling: because of chance, you have to try things repeated
 - Difficulty: learning can be much harder than solving a known MDP

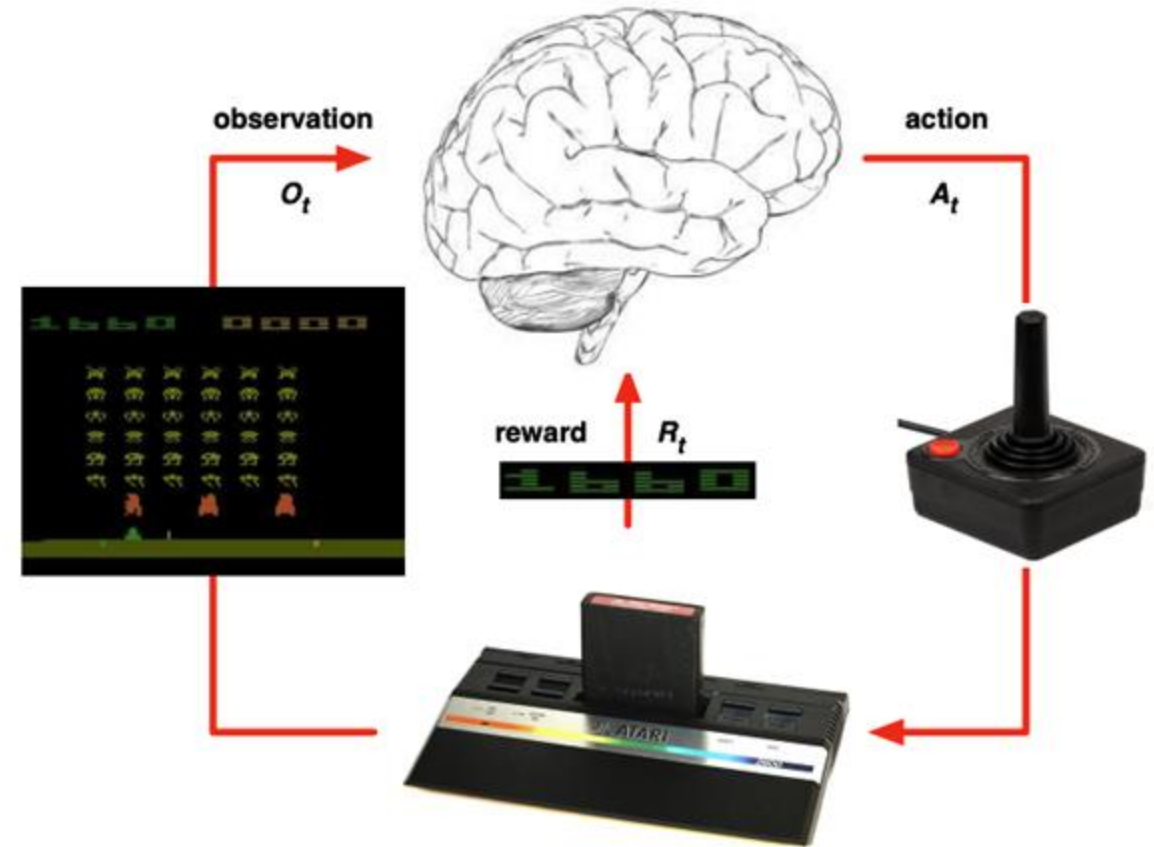
Atari Example: Planning

- Assume rules of the game are known
- Can query emulator
 - perfect model inside agent's brain
- If I take action a from state s :
 - what would the next state be?
 - what would the score be?
- Plan ahead to find optimal policy
 - E.g. tree search



Atari Example: Reinforcement Learning

- Rules of the game are unknown
- Learn directly from interactive game-play
- Pick actions on joystick, see pixels and scores

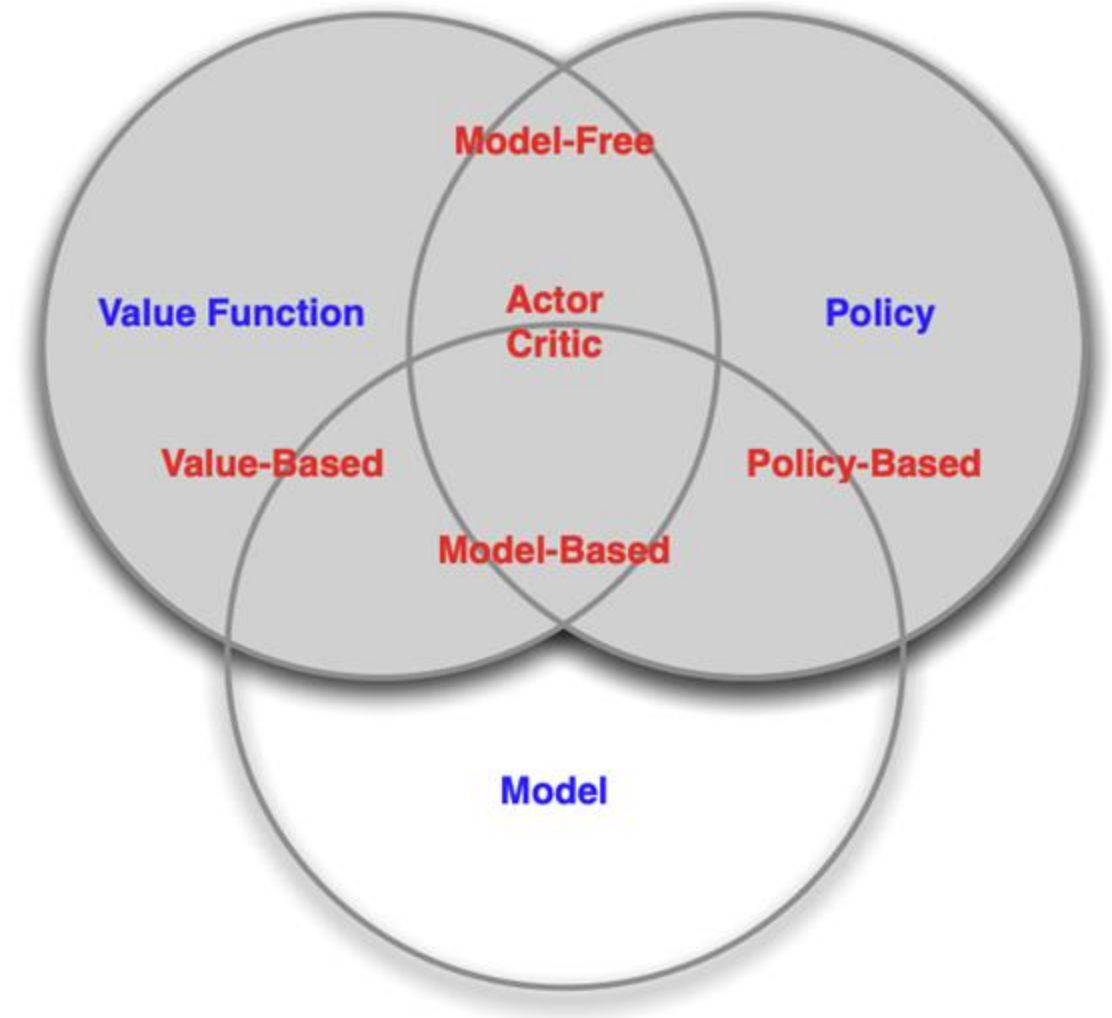


Towards Learning

- Now let's focus on reinforcement learning, where the environment is unknown. How can we apply learning?
 - 1. Learn a model of the environment (state transition model & reward function), and do planning in the model (i.e. **model-based reinforcement learning**)
 - You already know how to do this in principle (solving a MDP), but it's very hard to get to work.
 - 2. Learn a **value function** (e.g. **Q-learning**)
 - 3. Learn a **policy** directly (e.g. **policy gradient**)
- How can we deal with extremely large state spaces?
 - **Function approximation**: choose a parametric form for the policy and/or value function (e.g. linear in features, neural net, etc.)

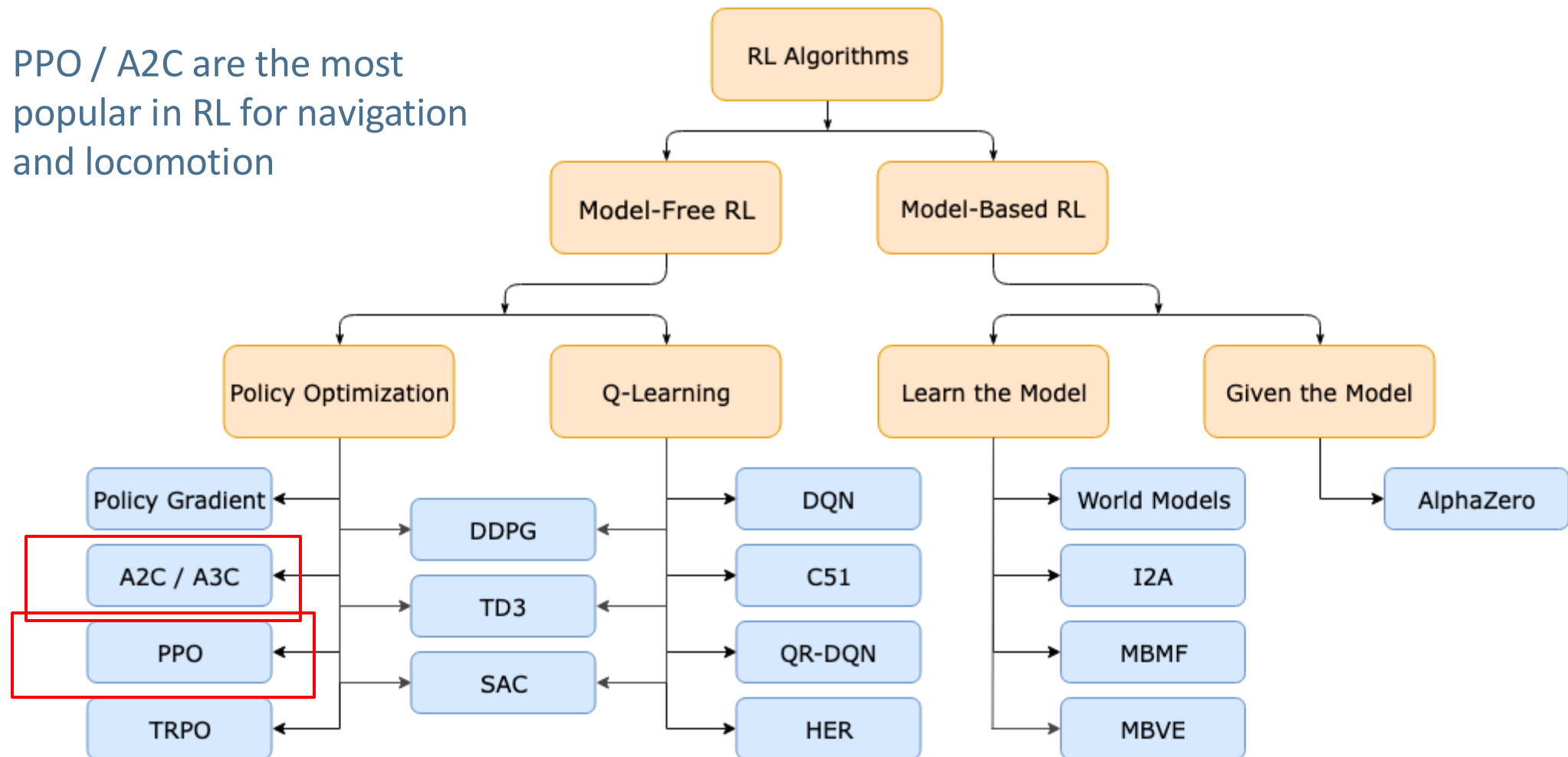
RL Agent Taxonomy

- Key ingredients
 - Policy
 - Value function
 - Model



A Taxonomy of RL Algorithms

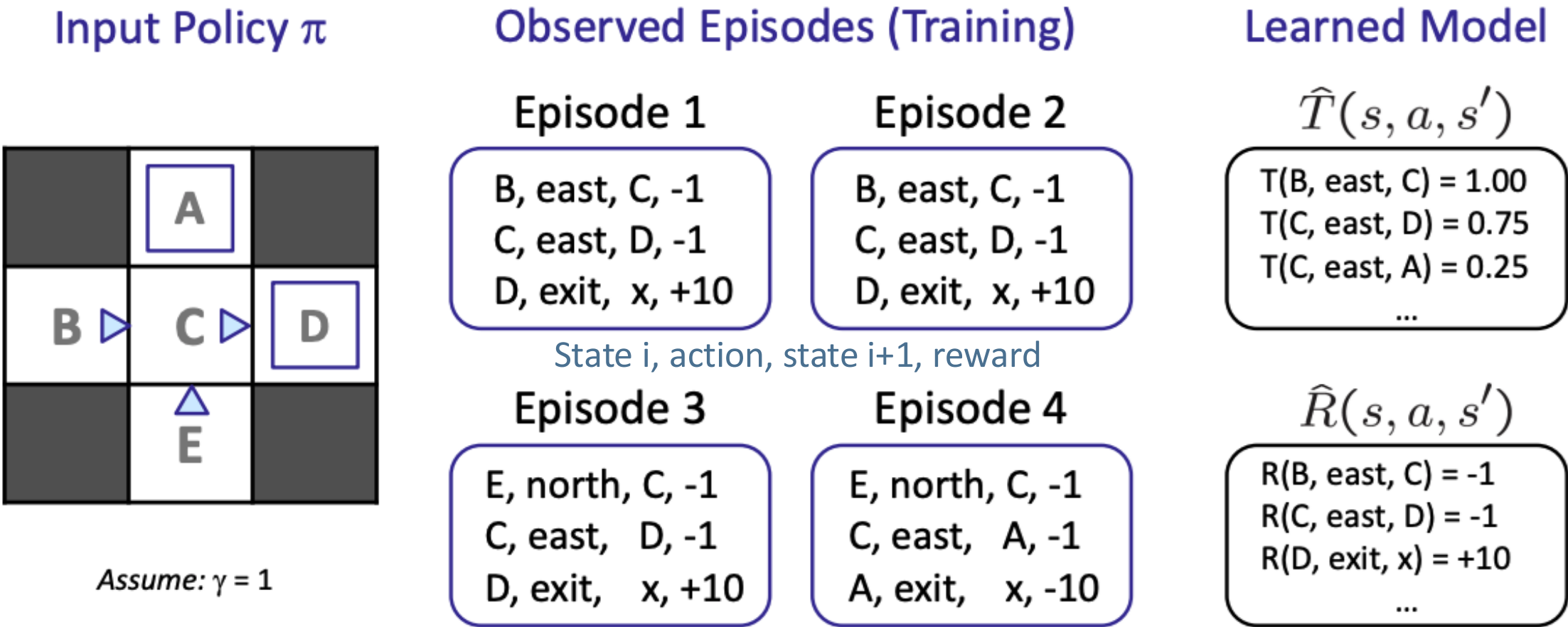
PPO / A2C are the most popular in RL for navigation and locomotion



Model-Based Learning

- Model-Based Idea:
 - Learn an approximate model based on experiences
 - Solve for values as if the learned model were correct
- Step 1: Learn empirical MDP model
 - Count outcomes s' for each s, a
 - Normalize to give an estimate of $\hat{T}(s, a, s')$
 - Discover each $\hat{R}(s, a, s')$ when we experience (s, a, s')
- Step 2: Solve the learned MDP
 - For example, use value iteration, as before

Model-Based Learning:



From the 4 episodes, starting from B (state), always go east (action) and end up in C (state). And this gets -1 reward

Model-Free: Monte Carlo Estimation

- Model-based is often impractical. We want to estimate the value function directly
- Recall the optimal Bellman equation for state-action value / Q function:

$$Q^*(s, a) = r(s, a) + \gamma \mathbb{E}_{\mathcal{P}(s' | s, a)} \left[\max_{a'} Q^*(s', a') \right]$$


To compute the expectations, we need the transition probabilities over all possible future states

- Problem: we need to know the dynamics (transition model) to evaluate the expectation

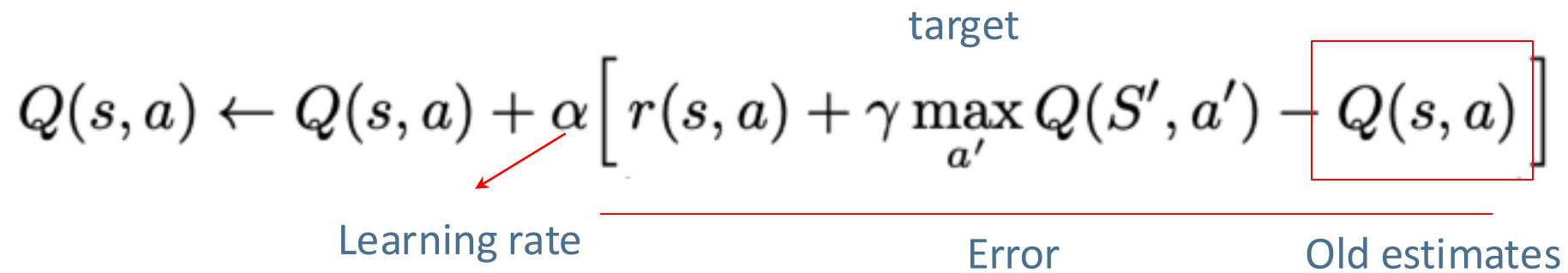
Model-Free: Monte Carlo Estimation

- Monte Carlo estimation of an expectation $\mu = \mathbb{E}[X]$: repeatedly sample X and update:
$$\mu \leftarrow \mu + \alpha(X - \mu)$$

- Idea: Apply Monte Carlo estimation to the Bellman equation
 - by sampling $S' \sim \mathcal{P}(\cdot | s, a)$
 - then updating till convergence: - Sample action a , get next state s'

$$Q(s, a) \leftarrow Q(s, a) + \alpha \left[r(s, a) + \gamma \max_{a'} Q(S', a') - Q(s, a) \right]$$

Learning rateErrorOld estimates



- This is called (Tabular) **Q-Learning**: replace expectation by samples
 - **Key idea: implicitly estimate the transitions via simulation**

Monte Carlo Estimation

- Problem 1: Every iteration of value iteration requires updating Q for every state
 - There could be lots of states
 - We only observe transitions for states that are visited
- Solution: Have the agent interact with the environment, and only update Q for the states that are actually visited
- Problem 2: We might never visit certain states if they don't look promising, so we'll never learn about them
- Solution: Have the agent sometimes take random actions so that it eventually visits every state
 - **ϵ -greedy policy**: a policy which picks action that maximizes $Q(s, a)$ with probability $1 - \epsilon$ and a random action with probability ϵ (Typical value: $\epsilon = 0.05$)

Q-Learning with ϵ -Greedy Policy

- Parameters: Learning rate α / Exploration parameter ϵ
- Initialize $Q(s, a)$ for all (s, a)
- The agent starts at state $S_0 \in \mathcal{S} \times \mathcal{A}$
- For time step $t = 0, 1, \dots$,
 - Choose A_t according to the ϵ -greedy policy, i.e.,

$$A_t \leftarrow \begin{cases} \operatorname{argmax}_{a \in \mathcal{A}} Q(S_t, a) & \text{with probability } 1 - \epsilon \\ \text{Uniformly random action in } \mathcal{A} & \text{with probability } \epsilon \end{cases}$$

- Take action A_t in the environment $\sim \mathcal{P}(\cdot | S_t, A_t)$
- The state changes from S_t to S_{t+1}
- Observe S_{t+1} and R_t
- Update the action-value function at state-action pair (S_t, A_t) :

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha \left[R_t + \gamma \max_{a' \in \mathcal{A}} Q(S_{t+1}, a') - Q(S_t, A_t) \right]$$

Exploration vs. Exploitation

- The ϵ -greedy is a simple mechanism for managing the exploration-exploitation tradeoff

$$\pi_{\epsilon}(S; Q) = \begin{cases} \operatorname{argmax}_{a \in \mathcal{A}} Q(S, a) & \text{with probability } 1 - \epsilon \\ \text{Uniformly random action in } \mathcal{A} & \text{with probability } \epsilon \end{cases}$$

- The ϵ -greedy policy ensures that most of the time (probability $1 - \epsilon$) the agent exploits its incomplete knowledge of the world by **chooses the best action** (i.e., corresponding to the highest action-value), but occasionally (probability ϵ) it **explores other actions**
- Without exploration, the agent may never find some good actions
- The ϵ -greedy is one of the simplest, but widely used, methods for trading-off exploration and exploitation. Exploration-exploitation tradeoff is an important topic of research

Examples of Exploration-Exploitation in the Real World

- Restaurant Selection
 - Exploitation: Go to your favourite restaurant
 - Exploration: Try a new restaurant
- Online Banner Advertisements
 - Exploitation: Show the most successful advert
 - Exploration: Show a different advert
- Oil Drilling
 - Exploitation: Drill at the best known location
 - Exploration: Drill at a new location
- Game Playing
 - Exploitation: Play the move you believe is best
 - Exploration: Play an experimental move

Off-Policy Learning

- Q-learning update again:

$$Q(S, A) \leftarrow Q(S, A) + \alpha \left[R + \gamma \max_{a' \in \mathcal{A}} Q(S', a') - Q(S, A) \right]$$

- Notice: **this update doesn't mention the policy anywhere**. The only thing the policy is used for is to determine which states are visited
- This means we can follow whatever policy we want (e.g. ϵ -greedy), and it still converges to the optimal Q-function. Algorithms like this are known as **off-policy algorithms**, and this is an extremely useful property
- Policy gradient is an **on-policy algorithm**. Encouraging exploration is much harder in that case

So for on-policy algorithm, you can only utilize the episodes of the current policy to do gradient updates (not from before or after)

Function Approximation

- So far, we've been assuming a **tabular** representation of Q : one entry for every state/action pair - keep a $|S| \times |A|$ table of $Q(s,a)$
- This is impractical to store for all but the simplest problems, and doesn't share structure between related states
- Solution: approximate Q using a parameterized function, e.g.
 - linear function approximation: $Q(s, a) = \mathbf{w}^\top \psi(s, a)$
 - Or compute Q with a neural net (non-linear approximation)
- Update Q using backpropagation:

$$\text{Target: } t \leftarrow r(\mathbf{s}_t, \mathbf{a}_t) + \gamma \max_{\mathbf{a}} Q(\mathbf{s}_{t+1}, \mathbf{a})$$

$$\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} + \alpha(t - Q(\mathbf{s}, \mathbf{a})) \nabla_{\boldsymbol{\theta}} Q(\mathbf{s}_t, \mathbf{a}_t)$$

Function Approximation

- Approximating Q with a neural net is a decades-old idea, but DeepMind got it to work really well on Atari games in 2013 (“**deep Q-learning**”)
- They used a very small network by today’s standards
- Main technical innovation: store experience into a **replay buffer**, and perform Q-learning using stored experience
 - Gains sample efficiency by separating environment interaction from optimization — don’t need new experience for every SGD update!

Deep Q-learning on Atari Games

- Mnih et al., Nature 2015. Human-level control through deep reinforcement learning
- Network was given raw pixels as observations
- Same architecture shared between all games
- After about a day of training on a particular game, often beat “human-level” performance (number of points within 5 minutes of play)



Value-based and Policy-based RL

- Value-based (Deep Q-Learning, **off-policy** method)
 - Learned value function
 - Implicit policy (e.g. ϵ -greedy)

$$V^{\pi}(s)$$

$$Q^{\pi}(s, a)$$

$$V^{*}(s)$$

$$Q^{*}(s, a)$$

$$\pi^{*}(a|s) = \begin{cases} 1 - \epsilon, & \text{if } a = \arg \max_a Q^{*}(s, a) \\ \epsilon, & \text{else} \end{cases}$$

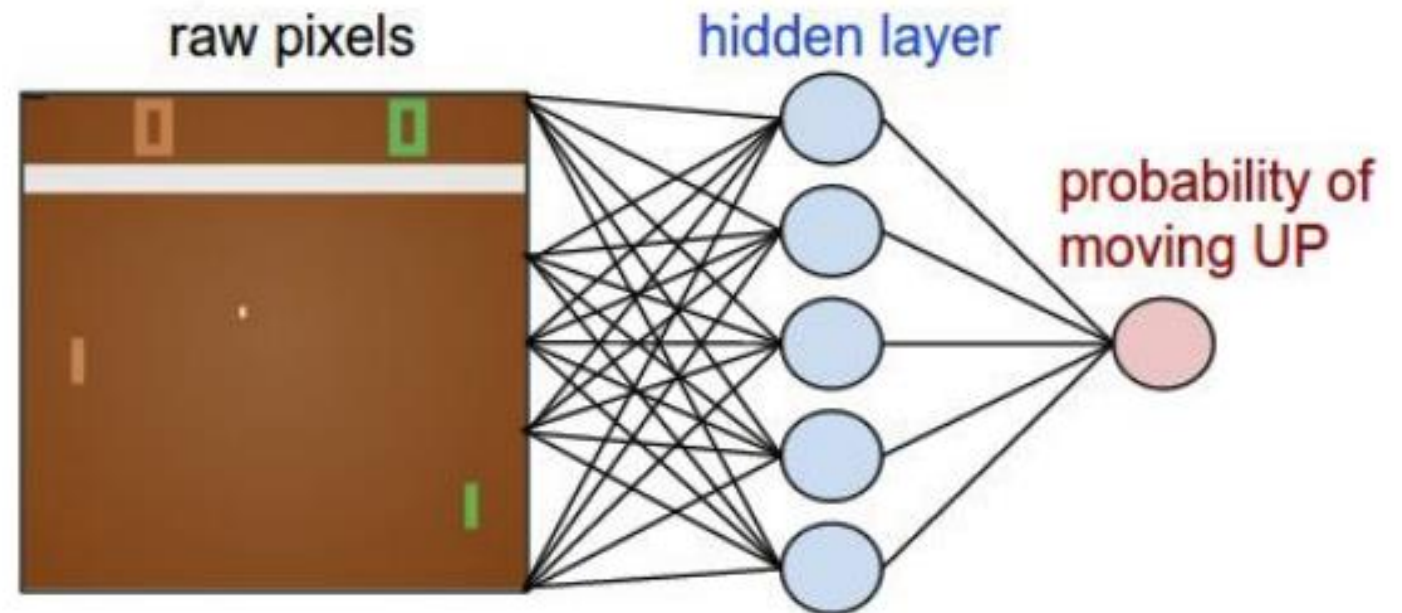
Exploitation: take the optimal action given optimal value

Exploration: there is some chance we will take a random action

Value-based and Policy-based RL

- Value-based (Deep Q-Learning, **off-policy** method)
 - Learned value function
 - Implicit policy (e.g. ϵ -greedy)
- Policy-based (all methods like PPO and A2C are **on-policy** methods)
 - No value function
 - **Learned policy**

$$\pi_{\theta}(s, a) = \mathbb{P}[a \mid s, \theta]$$



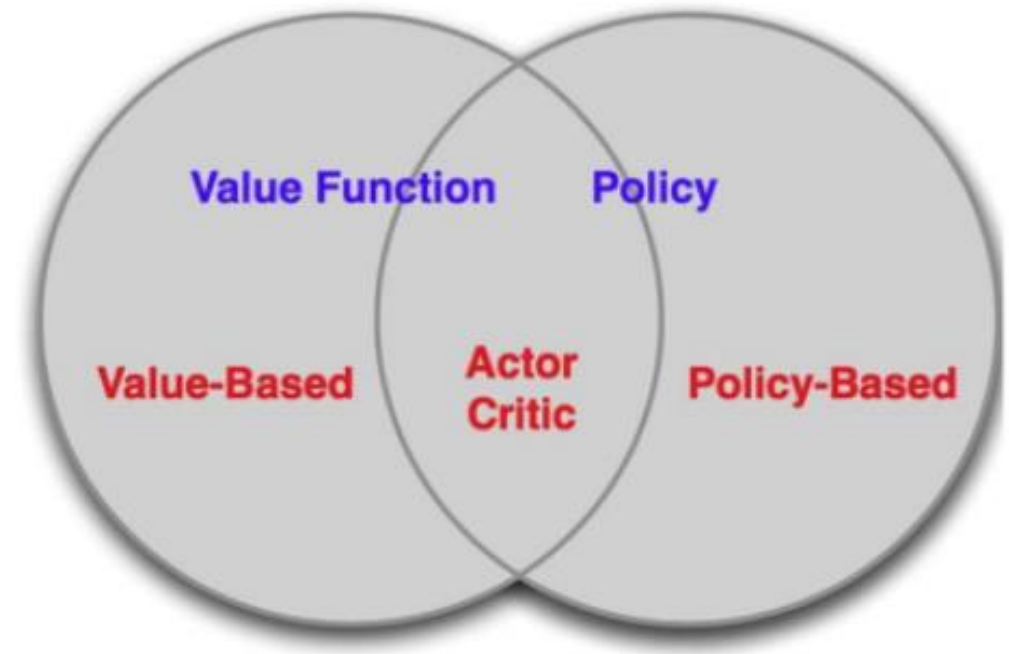
Given the state: an image, learn the policy that maps image to actions

Directly learning the policy - Why?

- Often the policy can be simpler than Q or V value function
 - E,g., robotic grasp
 - $Q(s,a)$ and $V(s)$ very high-dimensional But policy could be just 'open/close hand'
- V: doesn't prescribe actions
 - Would need dynamics model
- Q: need to be able to efficiently solve $\operatorname{argmax}_{a \in \mathcal{A}} Q(S, a)$ to get actions
 - Challenge for continuous / high-dimensional action spaces

Value-based and Policy-based RL

- Value-based
 - Learned value function
 - Implicit policy (e.g. ϵ -greedy)
- Policy-based
 - No value function
 - Learned policy
- **Actor-Critic**
 - Learned value function
 - Learned policy



Value-based and Policy-based RL

- Conceptually:

Policy-based

Optimize what you care about

Value-based

Indirect, exploit the problem structure, self-consistency

- Empirically:

More compatible with rich architectures (including recurrence)

More compatible with exploration and off-policy learning

More versatile

More sample-efficient when they work

More compatible with auxiliary objectives

Actor-critic methods

- A better baseline: push up the probability of an action from a state
 - if this action was better than the expected value of what we should get from that state (this action has an **advantage**)
- Recall: Q and V - action value and state value functions!
- We are happy with an action a in a state s **if $Q(s,a) - V(s)$ is large** Otherwise we are unhappy with an action if it's small.
- Using this, we get the estimator:

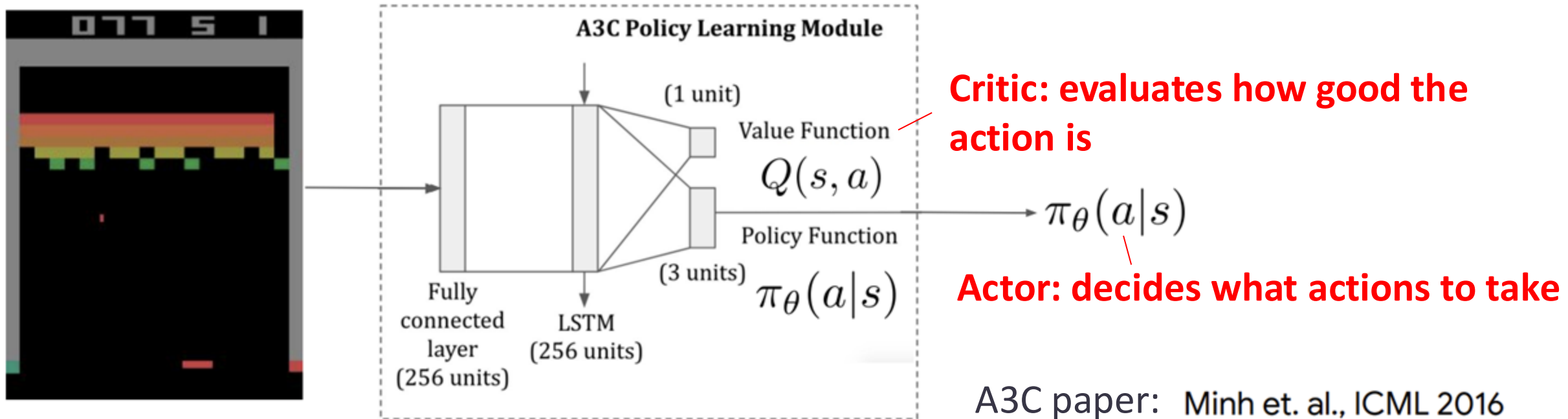
$$\nabla_{\theta} J(\theta) \approx \sum_{t \geq 0} \underbrace{(Q^{\pi_{\theta}}(s_t, a_t) - V^{\pi_{\theta}}(s_t))}_{\text{Advantage function: estimate of the relative value of the selected action}} \nabla_{\theta} \log \pi_{\theta}(a_t | s_t)$$

Advantage function: estimate of the relative value of the selected action

So now we have a way to “scale” the gradients based on value estimates in a trajectory (before we consider all positive/negative) with the help of the value function

Actor-critic methods

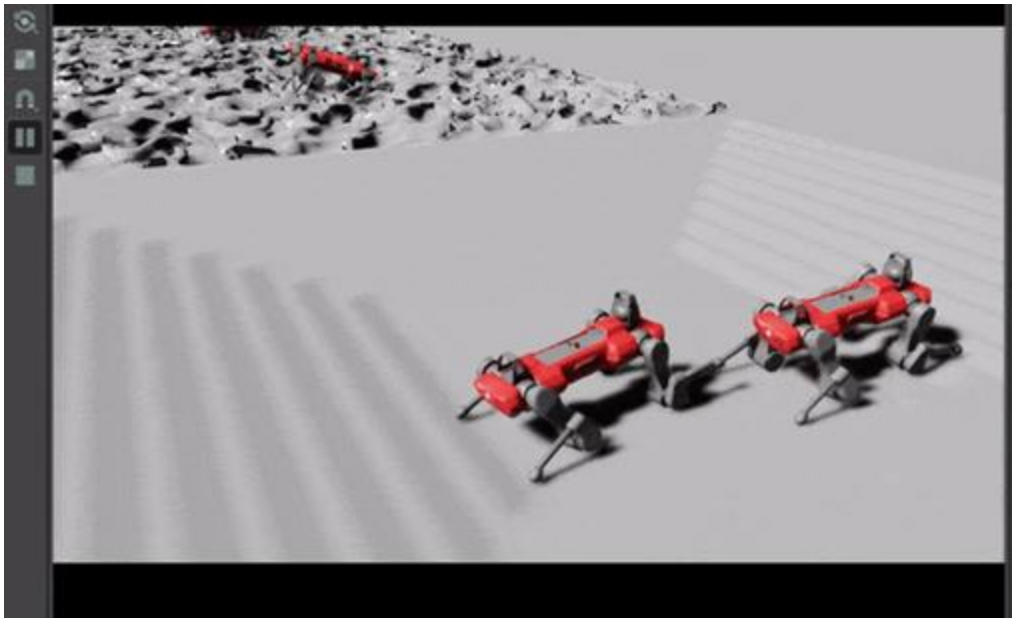
- Problem: we don't know Q and V - can we learn them?
- Yes, using Q-learning! We can combine Policy Gradients and Q-learning by training both **an actor (the policy)** and **a critic (the Q function)**



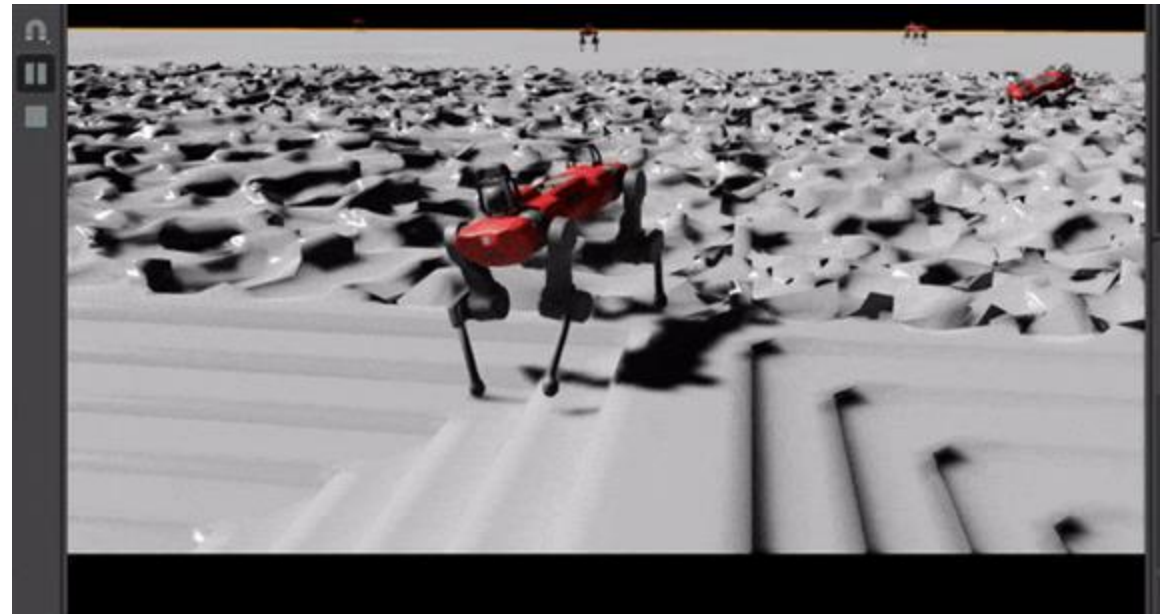
Actor-critic methods

Proximal Policy Optimization (PPO)

- PPO is used in many recent navigation and manipulation task (the go-to optimizer)
 - And locomotion - four legged control policy training



Single-GPU 10 minutes of training



Single-GPU 2 hours of training

*Example code from <https://github.com/NVIDIA-Omniverse/OmniverseGymEnvs/tree/main>

*Also see code here for PPO: https://github.com/Denys88/rl_games/blob/master/rl_games/common/common_losses.py#L39

Summary of RL methods

Value Based

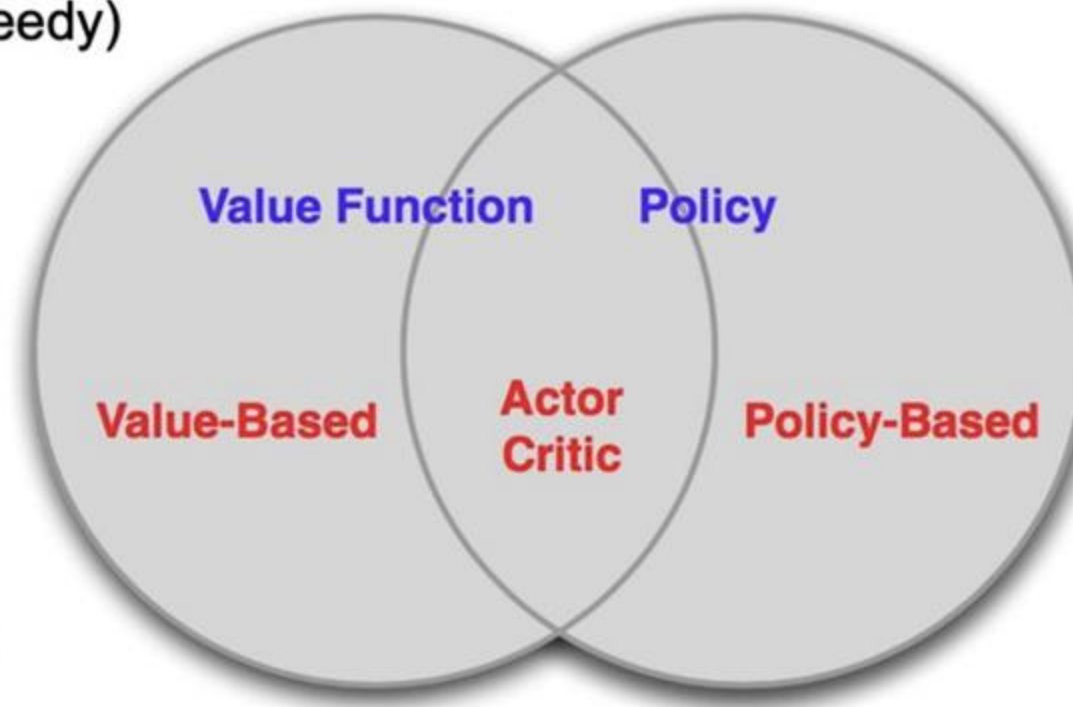
- Value iteration
- Policy iteration
- (Deep) Q-learning
- Learned Value Function
- Implicit policy (e.g. ϵ -greedy)

Policy Based

- Policy gradients
- No Value Function
- Learned Policy

Actor-Critic

- Actor (policy)
- Critic (Q-values)
- Learned Value Function
- Learned Policy





Thanks for your attention!

Changhao Chen
HKUST (GZ)

changhaochen@hkust-gz.edu.cn

Homepage: [changhao-chen@github.io](https://github.com/changhao-chen)